Complete Flow of Your Project

1. main.jsx

This file starts the React app.

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')).render(
 <React.StrictMode>
  <App />
 </React.StrictMode>
)
```

Flow

➞ React starts from main.jsx
➞ It loads App.jsx

---

2. App.jsx

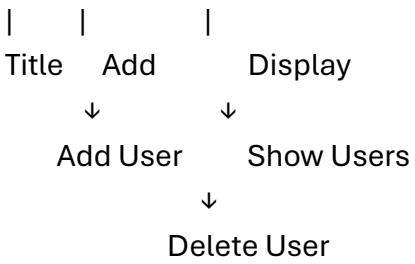
This is the MAIN component.

It controls:

- Data storage
- Add user
- Delete user
- Sending data to child components

Full Flow Diagram

```
main.jsx
  ↓
App.jsx
  ↓
--------------------------------
```



---

3. App.jsx Code Explanation

const [datas, setdatas] = useState([])

What happens?

- datas = stores all users
- setdatas = updates users

Example:

```
datas = [
 {
  name:"Rahul",
  email:"rahul@gmail.com"
 }
]
```

---

4. Add User Flow

Step 1

User enters data in Add.jsx

Input Field → Submit Button

Step 2

Add.jsx sends data to App component

addUser(data)

Step 3

In App.jsx

```
const addUser = (data) => {
  setdatas([...datas, data])
}
```

What this does?

Old data:

```
[
 {name:"A"}
]
```

New data:

```
{name:"B"}
```

After update:

```
[
 {name:"A"},
 {name:"B"}
]
```

---

5. Display Flow

App.jsx sends data to Display.jsx

```
<Display datas={datas} />
```

Now Display.jsx receives:

datas

Then loop using map():

```
datas.map((data, index) => (
  <div key={index}>
    {data.name}
  </div>
))
```

---

6. Delete Flow

Step 1

Click delete button in Display.jsx

```
<button onClick={() => deleteUser(index)}>
  Delete
</button>
```

Step 2

deleteUser() runs in App.jsx

```
const deleteUser = (index) => {
  const newArray = datas.filter((_, i) => i !== index)
  setdatas(newArray)
}
```

How Delete Works

Suppose:

```
[
  "A",
  "B",
  "C"
]
```

Delete index 1

i !== 1

New array:

```
[
  "A",
  "C"
]
```

Complete Project Structure

```
src/
│
├── main.jsx
├── App.jsx
│
├── components/
│   ├── Title.jsx
│   ├── Add.jsx
│   └── Display.jsx
```

Data Passing Flow

Parent → Child

```
<App>
  ↓ props
<Add addUser={addUser} />
```

Child → Parent

Add.jsx calls:

addUser(data)

This sends data back to App.

Why App.jsx is Important

App.jsx is called:

✅ Parent Component
✅ Main Component
✅ State Manager

Because all data is controlled there.

Simple Real-Life Example

App.jsx = Boss

Add.jsx = Employee who adds data

Display.jsx = Employee who shows data

All control stays with Boss (App.jsx).

edit ka flow

Edit Flow in React CRUD App

When you want to edit a user, the flow becomes like this 👇

Display User
↓
Click Edit Button
↓
Send Selected User Data to App.jsx
↓
Store Data in State
↓
Pass Data to Add.jsx
↓
Input Fields Filled Automatically
↓
Update User
↓
Replace Old Data
↓
Display Updated User

Complete Edit Flow Step-by-Step

1. Create Edit State in App.jsx

```
const [editIndex, setEditIndex] = useState(null)
const [editData, setEditData] = useState(null)
```

Why?

editIndex

Stores which user is being edited.

Example:

1

Means second user is editing.

---

editData

Stores complete user data.

Example:

```
{
 name: "Rahul",
 email: "rahul@gmail.com"
}
```

---

2. Click Edit Button

Inside Display.jsx

```
<button onClick={() => editUser(data, index)}>
 Edit
</button>
```

Flow

User clicks:

Edit Button

Then sends:

data + index

to App.jsx

---

3. Create Edit Function in App.jsx

```
const editUser = (data, index) => {
 setEditData(data)
 setEditIndex(index)
}
```

What Happens?

Suppose:

```
{
 name:"Amit",
 email:"amit@gmail.com"
}
```

gets stored in:

editData

and index:

2

stored in:

editIndex

---

4. Send Edit Data to Add.jsx

```
<Add
 addUser={addUser}
 editData={editData}
 updateUser={updateUser}
/>
```

---

5. Auto Fill Inputs

Inside Add.jsx

Use useEffect

```
useEffect(() => {
  if (editData) {
    setFormData(editData)
  }
}, [editData])
```

What Happens?

When editData changes:

```
{
  name:"Rahul",
  email:"rahul@gmail.com"
}
```

Input fields automatically fill.

---

6. Update User Function

In App.jsx

```
const updateUser = (updatedData) => {

  const updatedArray = datas.map((item, index) =>
    index === editIndex ? updatedData : item
  )

  setdatas(updatedArray)

  setEditIndex(null)
  setEditData(null)
}
```

How Update Works

Old Array:

[
{name:"A"},
{name:"B"},
{name:"C"}
]

Edit index:

1

Updated Data:

{name:"Updated B"}

New Array:

[
{name:"A"},
{name:"Updated B"},
{name:"C"}
]

---

7. Submit Button Logic

Inside Add.jsx

```
if (editData) {
  updateUser(formData)
} else {
  addUser(formData)
}
```

Final Edit Flow Diagram

Display.jsx
↓
Click Edit
↓
App.jsx
(setEditData)
(setEditIndex)

↓
Add.jsx
(useEffect fills form)
↓
Update Button
↓
updateUser()
↓
setdatas()
↓
Updated UI

Full CRUD Flow

CREATE → addUser()

READ → Display.jsx

UPDATE → editUser() + updateUser()

DELETE → deleteUser()

easy way and shortly

Simple Edit Flow in React CRUD

Step 1 → Click Edit Button

Inside Display.jsx

```
<button onClick={() => editUser(data, index)}>
 Edit
</button>
```

Step 2 → Send Data to App.jsx

```
const editUser = (data, index) => {
 setFormData(data)
 setEditIndex(index)
}
```

Step 3 → Fill Input Fields

Pass data to Add.jsx

<Add formData={formData} />

Inputs automatically show old data.

Step 4 → Update Data

```
const updateUser = (newData) => {

  const updated = datas.map((item, i) =>
    i === editIndex ? newData : item
  )

  setdatas(updated)
}
```

Easy Flow Diagram

Edit Button
  ↓
Send Old Data
  ↓
Fill Form
  ↓
Change Data
  ↓
Update Array
  ↓
Show Updated Data